

Leader Election

Floodmax ed LCR

Roberto Marino, PhD

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

18 maggio 2026



UniMe
1548

Problema 1.55 — Leader Election

Fino ad ora abbiamo studiato algoritmi in cui tutti i processori cooperano verso un obiettivo comune (broadcast, shortest path). Esiste però una classe di problemi in cui i processori devono *coordinarsi* per eleggere un rappresentante unico: il **leader**.

Problema 1.55 — Leader Election

Ogni processore ha una variabile `leader`, inizialmente posta a `unknown`. Si dice che un leader è eletto quando **esattamente un** processore ha `leader = true` e tutti gli altri hanno `leader = false`.
Eleggere un leader.

Perché serve un leader?

- ▶ Coordinare accessi a risorse condivise
- ▶ Iniziare algoritmi che richiedono un nodo speciale (es. broadcast)
- ▶ Rilevare guasti e nominare un sostituto

La soluzione naturale: il massimo UID.

Poiché ogni processore ha un UID unico, la convenzione più semplice è eleggere il processore con l'UID più grande. Il problema si riduce quindi a: come fa ogni nodo a scoprire il massimo UID della rete *senza una memoria condivisa*?

Idea. In ogni round ogni nodo propaga ai propri out-neighbor il massimo UID che ha ricevuto fino a quel momento (**max-id**). Dopo **diam($\mathcal{S}$) round** — il numero massimo di hop tra due nodi qualsiasi nella rete — ogni nodo ha ricevuto il massimo UID globale. A quel punto confronta **max-id** con il proprio **my-id**: se coincidono, è il leader.

Stato del processore:

$w = (\text{my-id}, \text{max-id}, \text{leader}, \text{round})$

- ▶ **my-id** $\in \{1, \dots, n\}$: UID proprio, immutabile
- ▶ **max-id** $\in \{1, \dots, n\}$: massimo UID visto finora
- ▶ **leader** $\in \{\text{true}, \text{false}, \text{unknown}\}$
- ▶ **round** $\in \{0, \dots, \text{diam}(\mathcal{S})\}$: contatore

Stato iniziale (per ogni nodo i):

- ▶ **my-id** $^{[i]} = i$
- ▶ **max-id** $^{[i]} = i$ (solo il proprio)
- ▶ **leader** $^{[i]} = \text{unknown}$
- ▶ **round** $^{[i]} = 0$

Assunzione critica

Ogni nodo deve conoscere $\text{diam}(\mathcal{S})$ (o un upper bound) a priori. Questa è una limitazione forte del Floodmax.

Specifica

Rete: $\mathcal{S} = (\{1, \dots, n\}, E_{\text{cmm}})$

Algoritmo: FLOODMAX

Alfabeto: $\mathbb{A} = \{1, \dots, n\} \cup \{\text{null}\}$

Stato: $w = (\text{my-id}, \text{max-id}, \text{leader}, \text{round})$

function msg(w, i)

```
if round < diam( $\mathcal{S}$ ) then
  return max-id
else return null
```

*Ogni nodo propaga il massimo che conosce
per esattamente $\text{diam}(\mathcal{S})$ round.*

function stf(w, y)

new-id := max{max-id, max identifier in y }

case

round < diam($\mathcal{S}$):

new-lead := unknown

round = diam($\mathcal{S}$) and max-id = my-id:

new-lead := true *sono il leader*

round = diam($\mathcal{S}$) and max-id > my-id:

new-lead := false *non sono il leader*

return (my-id, new-id, new-lead, round+1)

Lemma 1.56 — Complessità del Floodmax

Per una rete $\mathcal{S}$ con spanning tree radicato in v :

- ▶ **Communication complexity:** $O(\text{diam}(\mathcal{S}) \cdot |E_{\text{cmm}}|)$
- ▶ **Time complexity:** $\text{diam}(\mathcal{S})$ round
- ▶ **Space complexity:** $\Theta(1)$ per nodo

Correttezza.

Dopo $\text{diam}(\mathcal{S})$ round, ogni nodo ha ricevuto il massimo UID della rete. Questo segue dal fatto che qualsiasi nodo è raggiungibile dalla sorgente in al più $\text{diam}(\mathcal{S})$ hop, e in ogni round il massimo UID si propaga di almeno un hop.

Solo il nodo con `my-id = max-id` pone `leader = true`: esattamente uno, poiché gli UID sono unici.

Limite del Floodmax.

Richiede che ogni nodo conosca $\text{diam}(\mathcal{S})$ a priori. In molte reti reali il diametro non è noto.

Se si usa un upper bound $D \geq \text{diam}(\mathcal{S})$, l'algoritmo rimane corretto ma spreca $D - \text{diam}(\mathcal{S})$ round inutili, aumentando la communication complexity.

Il **LCR algorithm** supera questa limitazione restringendosi agli anelli, dove il diametro non

Il **LCR algorithm** (Le Lann–Chang–Roberts) risolve la leader election su **reti ad anello**: un digrafo in cui ogni nodo ha esattamente un in-neighbor e un out-neighbor, formando un ciclo diretto.

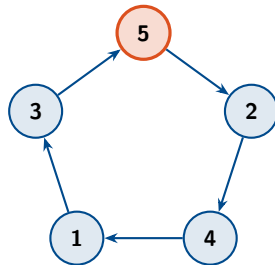
Vantaggio rispetto a Floodmax.

LCR non richiede che i nodi conoscano il diametro (o la dimensione) della rete. L'algoritmo termina autonomamente quando il leader riceve il proprio UID dall'in-neighbor.

Idea.

Ogni nodo manda il proprio UID in avanti nell'anello. Quando riceve un UID dal vicino lo confronta con il massimo che conosce (**max-id**):

- ▶ **maggiore del proprio**: lo ritrasmette
- ▶ **minore del proprio**: lo scarta, smette di trasmettere
- ▶ **uguale al proprio**: il proprio UID ha percorso l'intero anello, dichiara se stesso leader



nodo 5: UID massimo, sarà il leader

Specifica

Rete: anello diretto di n nodi

Algoritmo: LCR

Alfabeto: $\mathbb{A} = \{1, \dots, n\} \cup \{\text{null}\}$

Stato: $w = (\text{my-id}, \text{max-id}, \text{leader}, \text{snd-flag})$

Stato iniziale (per ogni nodo i):

- ▶ $\text{my-id}^{[i]} = i$
- ▶ $\text{max-id}^{[i]} = i$
- ▶ $\text{leader}^{[i]} = \text{unknown}$
- ▶ $\text{snd-flag}^{[i]} = \text{true}$

Tutti partono pronti a trasmettere il proprio UID.

```
function msg( $w, i$ )
```

```
  if  $\text{snd-flag} = \text{true}$  then  
    return  $\text{max-id}$   
  else return  $\text{null}$ 
```

Un nodo trasmette solo quando ha qualcosa di nuovo da dire: quando riceve un UID maggiore del proprio, lo propaga subito. Altrimenti tace ($\text{snd-flag} = \text{false}$).

Notare che snd-flag sostituisce il contatore round di Floodmax: la terminazione emerge dall'algoritmo stesso, senza conoscere la dimensione della rete.

```
function stf( $w, y$ )
```

```
  case
```

```
     $y$  contiene solo null oppure  $\max(y) < \text{my-id}$ :
```

```
      nessun UID utile ricevuto
```

```
      new-id := max-id
```

```
      new-lead := leader
```

```
      new-snd-flag := false
```

```
     $\max(y) = \text{my-id}$ :
```

```
      il mio UID ha percorso l'intero anello
```

```
      new-id := max-id
```

```
      new-lead := true
```

```
      new-snd-flag := false
```

```
     $\max(y) > \text{my-id}$ :
```

```
      ricevuto UID maggiore: propagarlo
```

```
      new-id :=  $\max(y)$ 
```

```
      new-lead := false
```

```
      new-snd-flag := true
```

Logica dei tre casi

Caso 1 — il nodo ha l'UID più grande tra quelli che ha visto. Non ha nulla di nuovo da propagare. Si mette in silenzio.

Caso 2 — il proprio UID è tornato indietro dopo aver percorso tutto l'anello senza essere scartato. Nessun altro UID nell'anello è più grande: questo nodo è il leader.

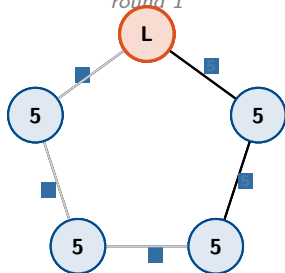
Caso 3 — è arrivato un UID più grande del proprio. Va ritrasmesso subito al prossimo vicino. `snd-flag` torna `true` per un altro round.

LCR — Esempio di esecuzione su anello di 5 nodi

round 0

round 1

round n



5 riceve il proprio UID: leader = true

Gli UID minori (1, 2, 3, 4) vengono scartati non appena incontrano un nodo con UID più grande. Solo il messaggio con valore 5 sopravvive e percorre l'intero anello. Quando torna al mittente, il nodo 5 si dichiara leader.

Il problema

Leader election: eleggere un unico processore con `leader = true`, tutti gli altri con `leader = false`. La convenzione è eleggere il nodo con UID massimo.

Floodmax

Idea: propaga il massimo UID per $\text{diam}(\mathcal{S})$ round; alla fine ogni nodo conosce il massimo globale.

Stato: (`my-id`, `max-id`, `leader`, `round`)

Limite: richiede $\text{diam}(\mathcal{S})$ noto a priori.

Complessità: $O(\text{diam} \cdot |E_{\text{cmm}}|)$ messaggi.

LCR

Idea: su anello, ogni nodo propaga solo UID maggiori del proprio; chi riceve il proprio UID è il leader.

Stato: (`my-id`, `max-id`, `leader`, `snd-flag`)

Vantaggio: non richiede conoscenza della dimensione della rete.

Complessità: $\Theta(n^2)$ messaggi, n round.

In entrambi gli algoritmi il leader emerge come effetto collaterale della propagazione del massimo UID nella rete.